



BITS Pilani
Pilani Campus

Unsupervised Deep Learning ZG533

Dr. Sugata Ghosal
sugata.ghosal@pilani.bits-pilani.ac.in



Session 2

These slides are prepared by the instructor, with grateful acknowledgement of many others who made their course materials freely available online.

Session Content



Factor Analysis

- Principal Component Analysis (PCA)
 - Randomized PCA
 - Incremental PCA
 - Kernel PCA
- Independent Component Analysis (ICA)
- Canonical Correlation Analysis (CCA)
- Locally Linear Embeddings (LLE)

Reading: R1: *Hands-On Machine Learning* ..., Chapter 8

Factor or Component Analysis



- Discover a new set of factors/dimensions/axes against which to represent, describe or evaluate the data
 - For more effective reasoning, insights, or better visualization
 - Reduce noise in the data
 - Typically a smaller set of factors: dimension reduction
 - Better representation of data without losing much information
 - Can build more effective data analyses on the reduced-dimensional space: classification, clustering, pattern recognition
- Factors are combinations of observed variables
 - May be more effective bases for insights, even if physical meaning is obscure
 - Observed data are described in terms of these factors rather than in terms of original variables/dimensions

Why Factor or Component Analysis?



- We study phenomena that can not be directly observed
 - ego, personality, intelligence in psychology
 - Underlying factors that govern the observed data
- We want to identify and operate with underlying latent factors rather than the observed data
 - E.g. topics in news articles
 - Transcription factors in genomics
- We want to discover and exploit hidden relationships
 - “beautiful car” and “gorgeous automobile” are closely related
 - So are “driver” and “automobile”
 - But does your search engine know this?
 - Reduces noise and error in results

Why Factor or Component Analysis?



- We have too many observations and dimensions
 - To reason about or obtain insights from
 - To visualize
 - Too much noise in the data
 - Need to “reduce” them to a smaller set of factors
 - Better representation of data without losing much information
 - Can build more effective data analyses on the reduced-dimensional space: classification, clustering, pattern recognition
- Combinations of observed variables may be more effective bases for insights, even if physical meaning is obscure

Principal Components Analysis (PCA)



- An exploratory technique used to reduce the dimensionality of the data set to 2D or 3D
- Can be used to:
 - Reduce number of dimensions in data
 - Find patterns in high-dimensional data
 - Visualize data of high dimensionality
- Example applications:
 - Face recognition
 - Image compression
 - Gene expression analysis

Principal Component Analysis

Goal: Find r -dim projection that best preserves variance

1. Compute mean vector μ and covariance matrix Σ of original points
2. Compute eigenvectors and eigenvalues of Σ
3. Select top r eigenvectors
4. Project points onto subspace spanned by them:

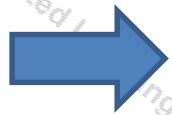
$$y = A(x - \mu)$$

where y is the new point, x is the old one,
and the rows of A are the eigenvectors

Covariance

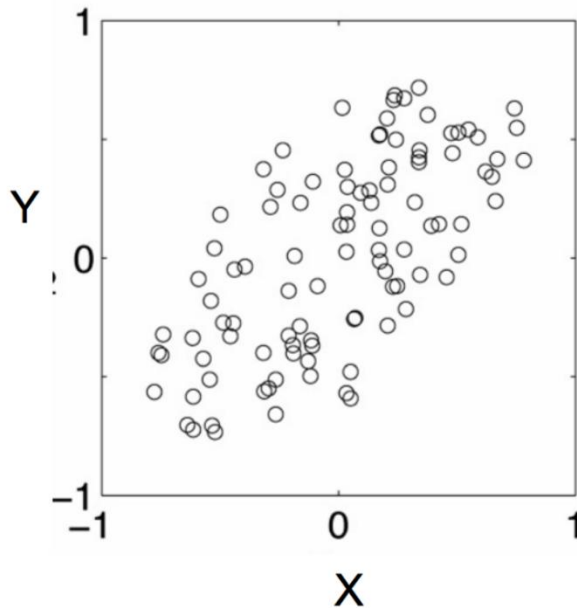


- Variance and Covariance:
 - Measure of the “spread” of a set of points around their center of mass(mean)
- Variance:
 - Measure of the deviation from the mean for points in one dimension
- Covariance:
 - Measure of how much each of the dimensions vary from the mean with **respect to each other**

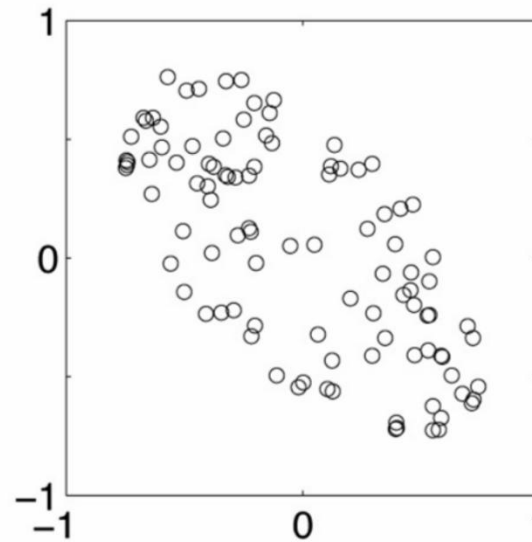


- **Covariance is measured between two dimensions**
- **Covariance sees if there is a relation between two dimensions**
- **Covariance between one dimension is the variance**

positive covariance



negative covariance



Positive: Both dimensions increase or decrease together **Negative:** While one increase the other decrease

Covariance

- Used to find relationships between dimensions in high dimensional data sets

$$q_{jk} = \frac{1}{N} \sum_{i=1}^N (X_{ij} - E(X_j)) (X_{ik} - E(X_k))$$



The Sample mean

Principal Component Analysis



- **Input:** $\mathbf{x} \in \mathbb{R}^D: \mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$
- **Set of basis vectors:** $\mathbf{u}_1, \dots, \mathbf{u}_K$
- **Summarize a D dimensional vector $\mathbf{x}$ with K dimensional feature vector $h(\mathbf{x})$**

$$h(\mathbf{x}) = \begin{bmatrix} \mathbf{u}_1 \cdot \mathbf{x} \\ \mathbf{u}_2 \cdot \mathbf{x} \\ \dots \\ \mathbf{u}_K \cdot \mathbf{x} \end{bmatrix}$$

Principal Component Analysis



$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_K]$$

Basis vectors are orthonormal

$$\mathbf{u}_i^T \mathbf{u}_j = 0$$

$$\|\mathbf{u}_j\| = 1$$

New data representation $h(\mathbf{x})$

$$z_j = \mathbf{u}_j \cdot \mathbf{x}$$

$$h(\mathbf{x}) = [z_1, \dots, z_K]^T$$

Principal Component Analysis

$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_K]$$

New data representation $h(\mathbf{x})$

$$h(\mathbf{x}) = \mathbf{U}^T \mathbf{x}$$

$$h(\mathbf{x}) = \mathbf{U}^T (\mathbf{x} - \mu_0)$$

Empirical mean



$$\mu_0 = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$$

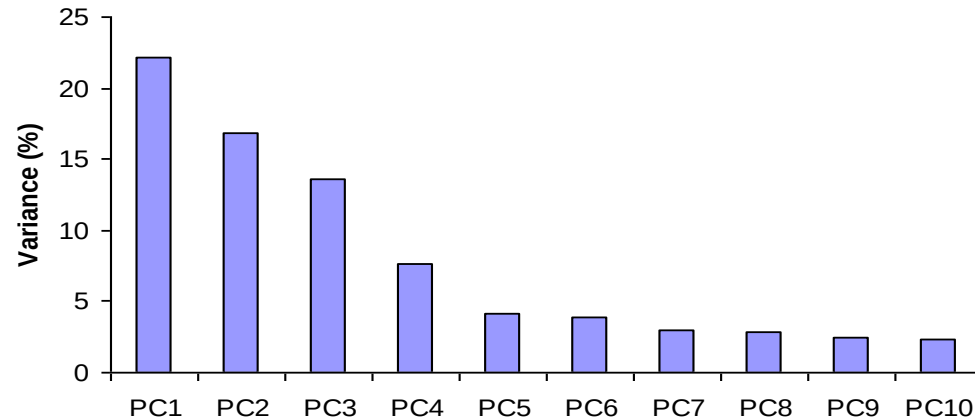
How Many PCs?

- For n original dimensions, correlation matrix is $n \times n$, and has up to n eigenvectors. So n PCs.
- Where does dimensionality reduction come from?

Dimensionality Reduction



Can *ignore* the components of lesser significance.



Do *lose some information*, but if eigenvalues are small, don't lose much

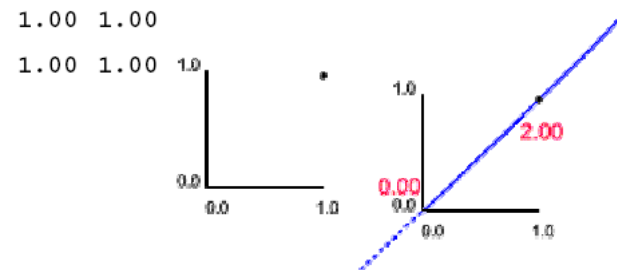
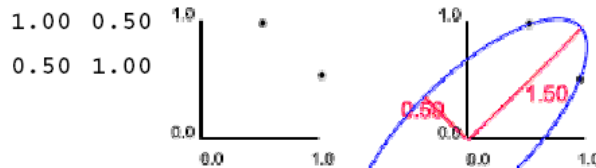
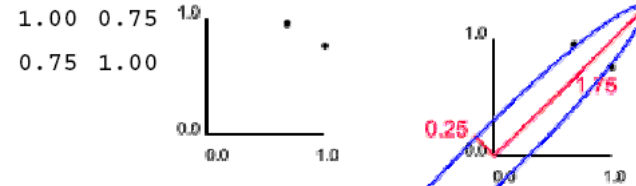
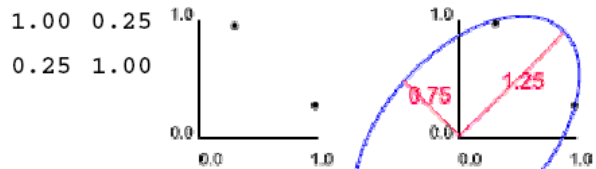
- n dimensions in original data
- calculate n eigenvectors and eigenvalues
- choose only the first p eigenvectors, based on their eigenvalues
- final data set has only p dimensions

Eigenvectors of a Correlation Matrix

innovate

achieve

lead



The Space of All Faces

- When viewed as vectors of pixel values, face images are extremely high-dimensional
 - 100x100 image = 10,000 dimensions
 - Slow and lots of storage
- But very few 10,000-dimensional vectors are valid face images
- We want to effectively model the subspace of face images



slide by Derek Hoiem

Eigenface Example

Mean: μ



Top eigenvectors: $u_1, \dots, u_k$



Representation

- Face $\mathbf{x}$ in “face space” coordinates:



$$\mathbf{x} \rightarrow [\mathbf{u}_1^T (\mathbf{x} - \mu), \dots, \mathbf{u}_k^T (\mathbf{x} - \mu)]$$

$$= w_1, \dots, w_k$$

- Reconstruction:



=



+


 $\hat{\mathbf{x}}$
 $\mathbf{x}$

=

 μ

+

 $w_1 u_1 + w_2 u_2 + w_3 u_3 + w_4 u_4 + \dots$

Reconstruction



$P = 4$



$P = 200$

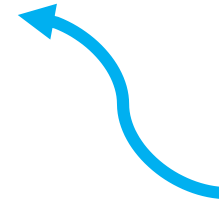


$P = 400$



After computing eigenfaces using 400 face images from ORL face database

Application: Image compression



**Original
Image**

- Divide the original 372x492 image into patches
 - Each patch is an instance that contains 12x12 pixels on a grid
- View each as a 144-D vector

PCA compression: 144D $\rightarrow$ 16D

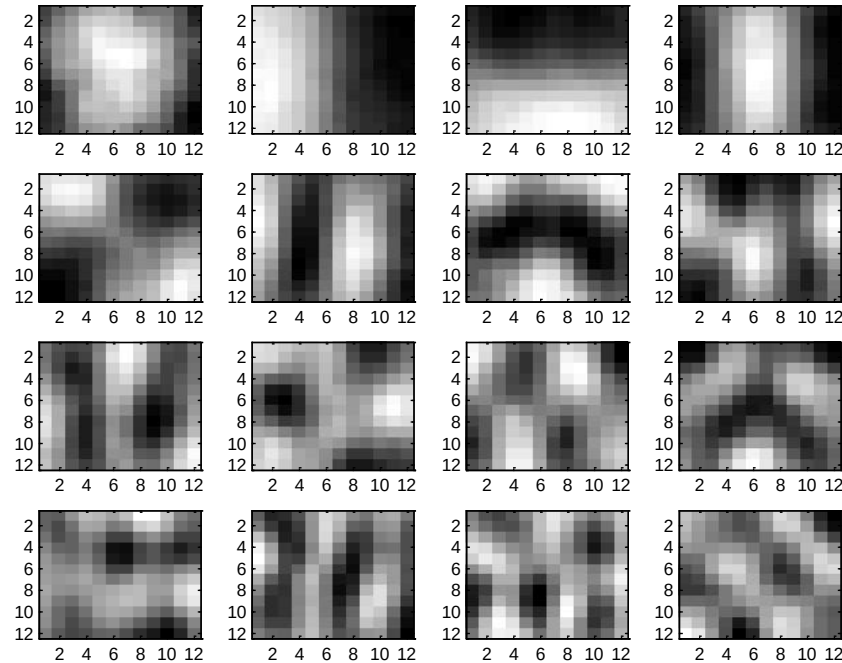
innovate

achieve

lead



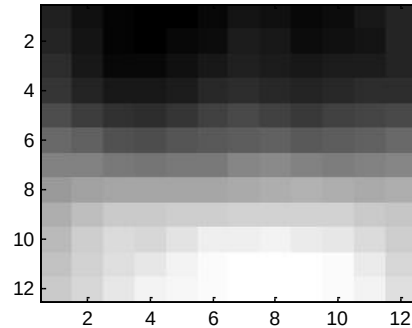
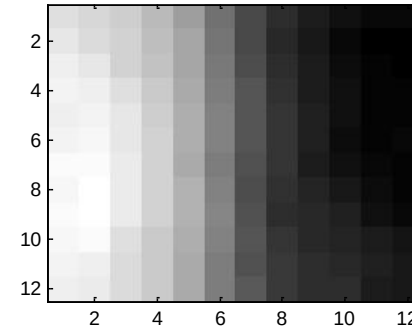
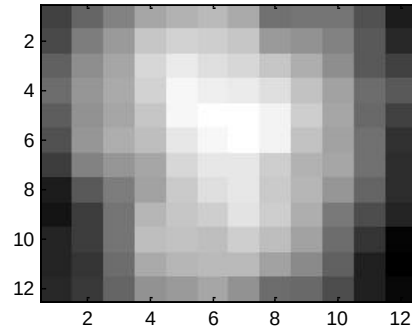
16 most important eigenvectors



PCA compression: 144D $\rightarrow$ 3D



3 most important eigenvectors



Randomized PCA

If you set the `svd_solver` hyperparameter to "randomized", Scikit-Learn uses a stochastic algorithm called *Randomized PCA* that quickly finds an approximation of the first d principal components. Its computational complexity is $O(m \times d^2) + O(d^3)$, instead of $O(m \times n^2) + O(n^3)$ for the full SVD approach, so it is dramatically faster than full SVD when d is much smaller than n :

```
rnd_pca = PCA(n_components=154, svd_solver="randomized")
X_reduced = rnd_pca.fit_transform(X_train)
```

By default, `svd_solver` is actually set to "auto": Scikit-Learn automatically uses the randomized PCA algorithm if m or n is greater than 500 and d is less than 80% of m or n , or else it uses the full SVD approach. If you want to force Scikit-Learn to use full SVD, you can set the `svd_solver` hyperparameter to "full".

For detailed math, try chatgpt.com

Incremental PCA



Divide Data into Batches: The data is divided into smaller batches that can comfortably fit into memory.

Partial Fit: Incremental PCA performs a 'partial fit' on each mini-batch rather than a 'fit' on the entire dataset.

Update Principal Components: After each partial fit, the principal components get updated, which means the final components are an approximation of the components that would have been obtained by running traditional PCA on the full dataset.

Data Transformation: Once all mini-batches have been processed, the data can be transformed into the reduced-dimensional space based on the final principal components.

Incremental PCA



Memory Efficiency: It's much more memory-efficient than traditional PCA.

Online Learning: Suitable for online machine learning models where you continue to receive new data points.

Limitations

Approximate Results: The principal components are approximations, which might be less accurate than the ones obtained from traditional PCA.

Computational Overhead: The continual update of components can result in increased computation time.

Incremental PCA



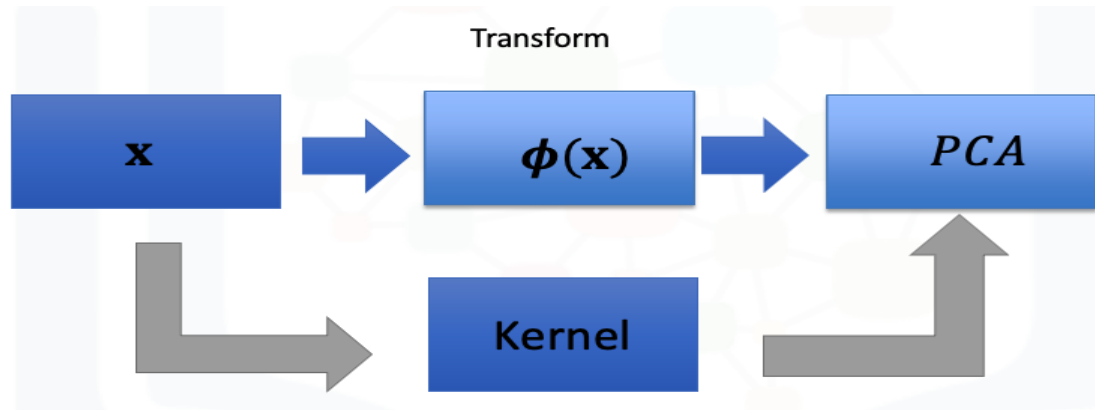
Highly useful in

- Real-time analytics
- Streamed data processing
- Large-scale machine learning tasks

For detailed math, try chatgpt.com

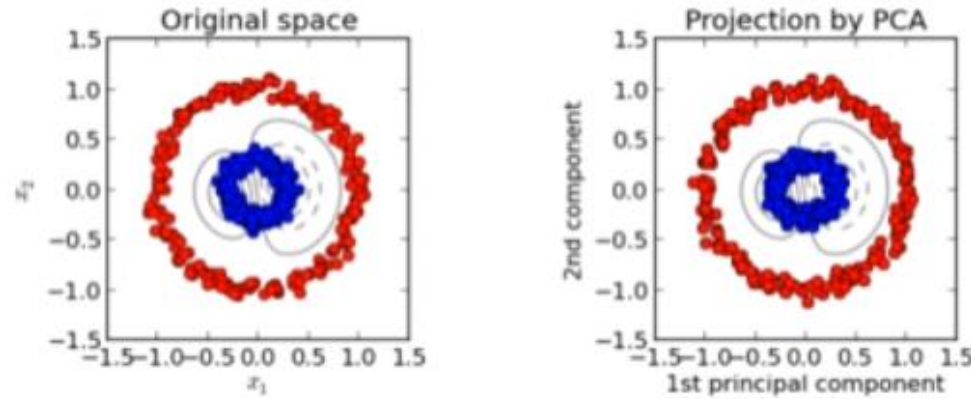
Kernel PCA

- An extension of PCA using kernel methods to map the data to a higher dimensional space, then performs PCA.
- The mapping to the higher dimensional space may be difficult to calculate, so the mapping is done via a kernel.

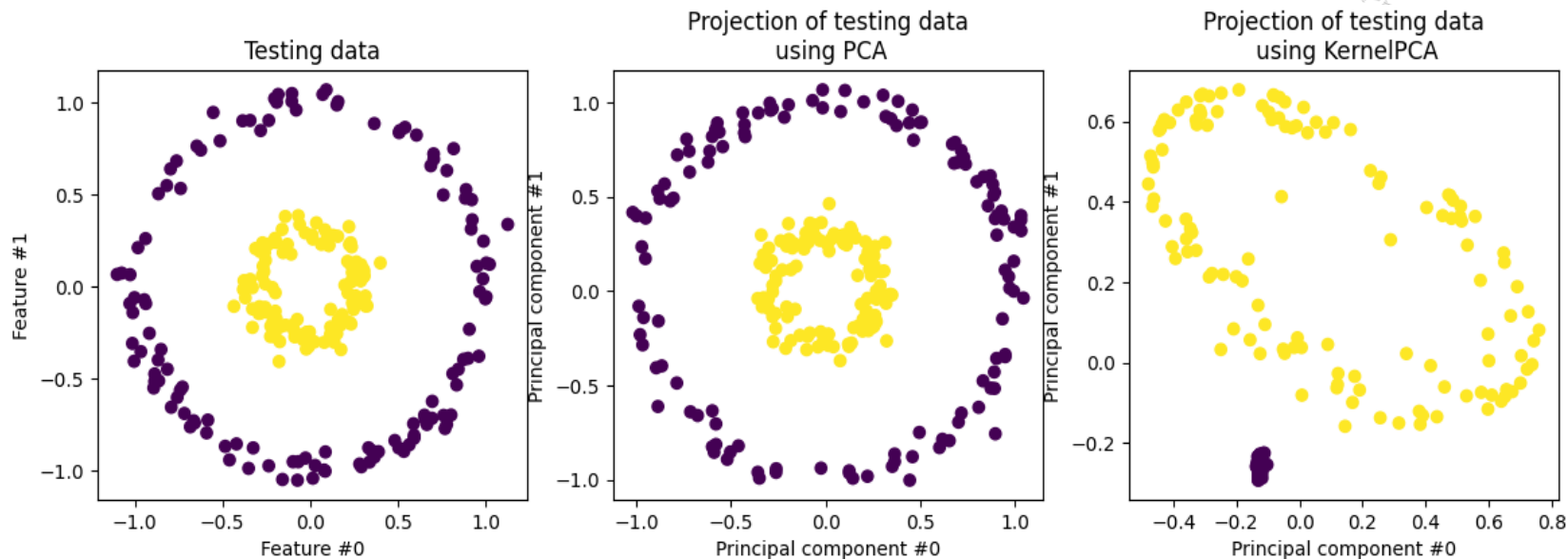


Kernel PCA

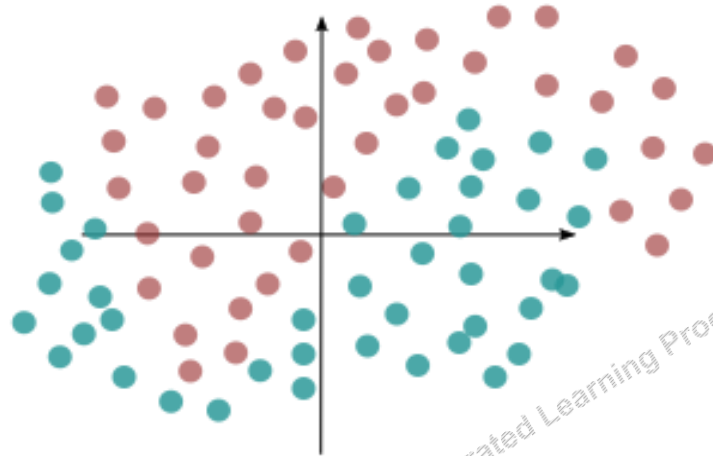
- PCA based features useful for classification only if the data are linearly separable.



Kernel PCA



- On the right hand side the classification boundary is more complicated, something that may look like a higher order polynomial, a non-linear function at any rate.



Kernel PCA



- Suppose the decision boundary is described by a third order polynomial

$$y = a + bx + cx^2 + dx^3.$$

- Plotting this function in the usual x, y plane. will produce a wavy line, similar to the decision boundary in the right-hand-side picture.
- Suppose instead we go to a higher dimensionality space in which the axes are

$$x, x^2, x^3 \text{ and } y$$

The “Kernel trick”

innovate

achieve

lead

Observation: PCA relies on eigenvectors of the sample covariance matrix $\mathbf{XX}^T$. This can be represented in the new feature space by the Kernel Matrix K , composed of dot products:

$$K = \Phi(X)\Phi(X)^T$$

Consequently we can *implicitly* model any nonlinear transformation $\Phi(\mathbf{x})$. The only requirement is that we can calculate the dot product $\Phi(\mathbf{x}_i)^T\Phi(\mathbf{x}_j)$ efficiently.

Kernel trick

Contains all pairwise evaluations of the dot product

$$\mathbf{K} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \dots & k(\mathbf{x}_1, \mathbf{x}_d) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \dots & k(\mathbf{x}_2, \mathbf{x}_d) \\ \vdots & \vdots & & \vdots \\ k(\mathbf{x}_d, \mathbf{x}_1) & k(\mathbf{x}_d, \mathbf{x}_2) & \dots & k(\mathbf{x}_d, \mathbf{x}_d) \end{bmatrix}$$

PCA's covariance matrix scales with the number of input dimensions n .

KPCA's kernel matrix scales with the number of datapoints d .

KPCA Recipe

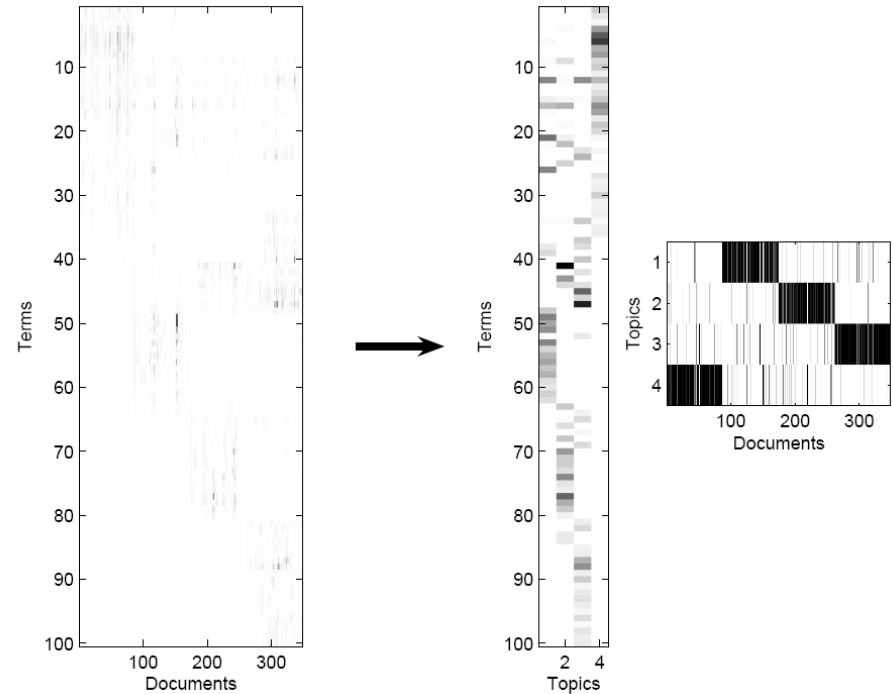
1. Pick a kernel function $k(\mathbf{x}_i, \mathbf{x}_j)$
2. Calculate the kernel matrix $\mathbf{K}$
3. Center the kernel matrix
$$\tilde{\mathbf{K}} = \mathbf{K} - 2(\mathbf{1}_{1/d})\mathbf{K} + (\mathbf{1}_{1/d})\mathbf{K}(\mathbf{1}_{1/d})$$

Matrix with entries $1/d$
4. Solve the eigenproblem $\tilde{\mathbf{K}}\alpha_i = \lambda_i\alpha_i$
5. Project the data to each new dimension j

$$y_j = \sum_{i=1}^d \alpha_{ij} k(\mathbf{x}_i, \mathbf{x}) \quad \text{for } j = 1, 2, \dots, m$$

PCs can be viewed as Topics

- In the sense of having to find quantities that are not observable directly
- Similarly, transcription factors in biology, as unobservable causal bridges between experimental conditions and gene expression



PCA is not the most sophisticated dimensionality reduction technique



- Dimensionality reduction is a useful technique for any classification/regression problem
 - Text retrieval can be seen as a classification problem
- There are many other dimensionality reduction techniques
 - Neural nets, support vector machines etc.
 -

Limitations of PCA

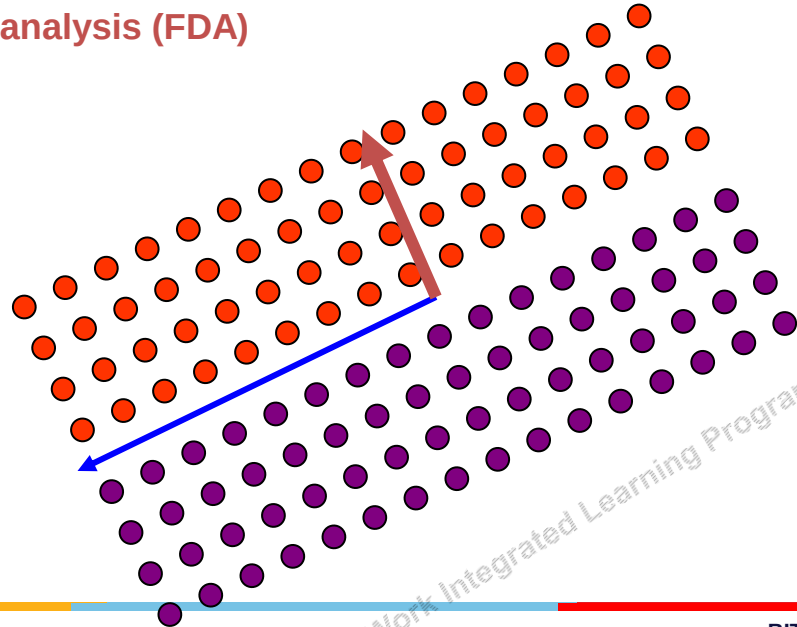
innovate

achieve

lead

Are the maximal variance dimensions the relevant dimensions for preservation?

- Relevant Component Analysis (RCA)
- Fisher Discriminant analysis (FDA)



Limitations of PCA

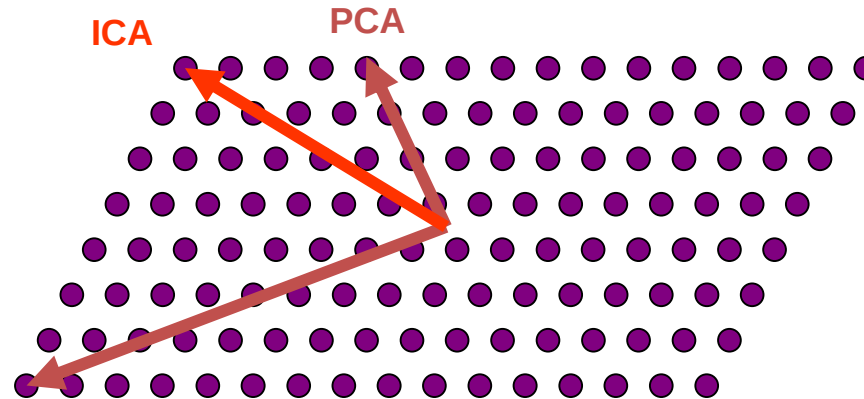
innovate

achieve

lead

Should the goal be finding independent rather than pair-wise uncorrelated dimensions

- Independent Component Analysis (ICA)



Limitations of PCA

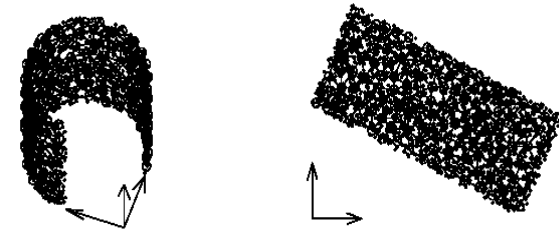


- The reduction of dimensions for complex distributions may need non linear processing
- Curvilinear Component Analysis (CCA)
 - Non linear extension of PCA
 - Preserves the proximity between the points in the input space i.e. local topology of the distribution
 - Enables to unfold some varieties in the input data
 - Keep the local topology

Example of representation using CCA

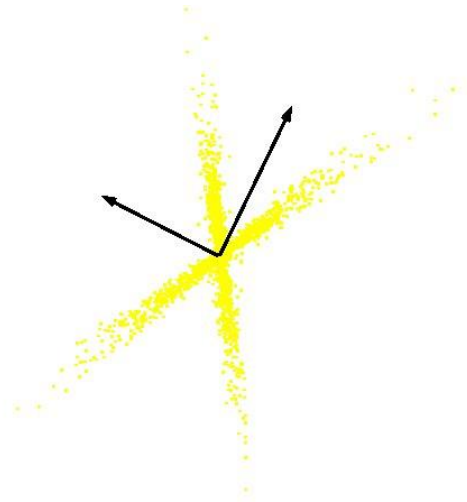


Non linear projection of a spiral

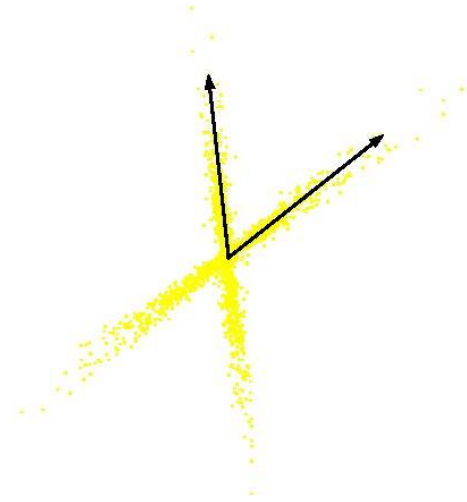


Non linear projection of a horseshoe

Independent Component Analysis (ICA)



PCA
(orthogonal coordinate)



ICA
(non-orthogonal coordinate)

Cocktail-party Problem



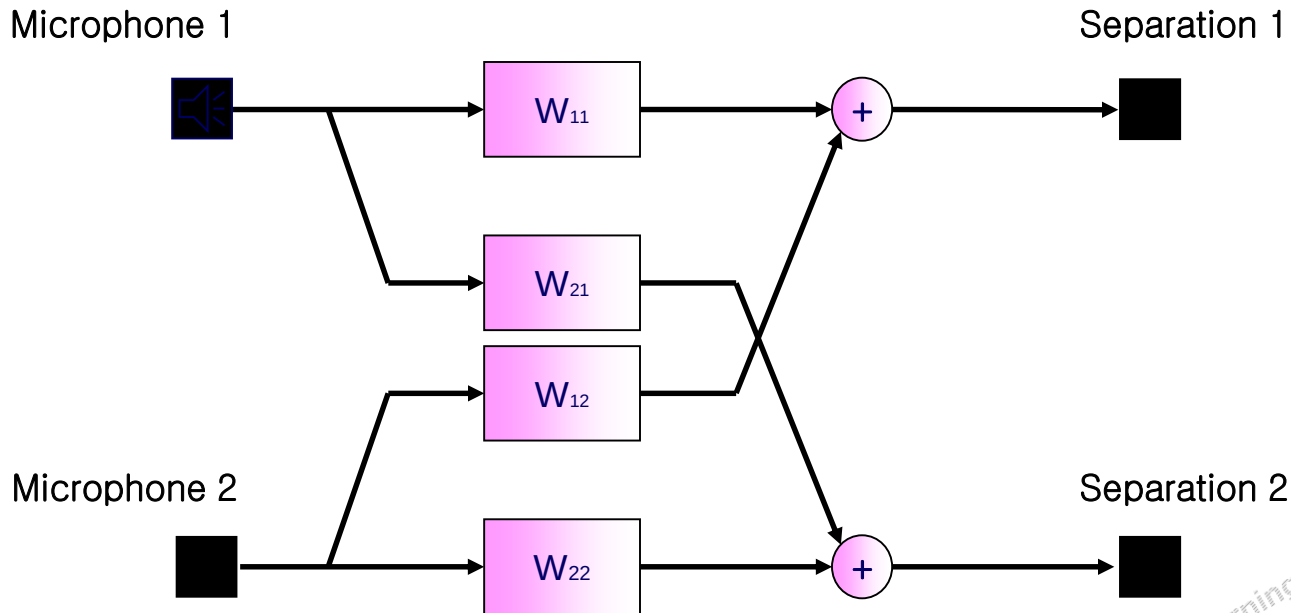
- Multiple sound sources in room (independent)
- Multiple sensors receiving signals which are mixture of original signals
- Estimate original source signals from mixture of received signals
- Can be viewed as Blind-Source Separation as mixing parameters are not known

BSS and ICA

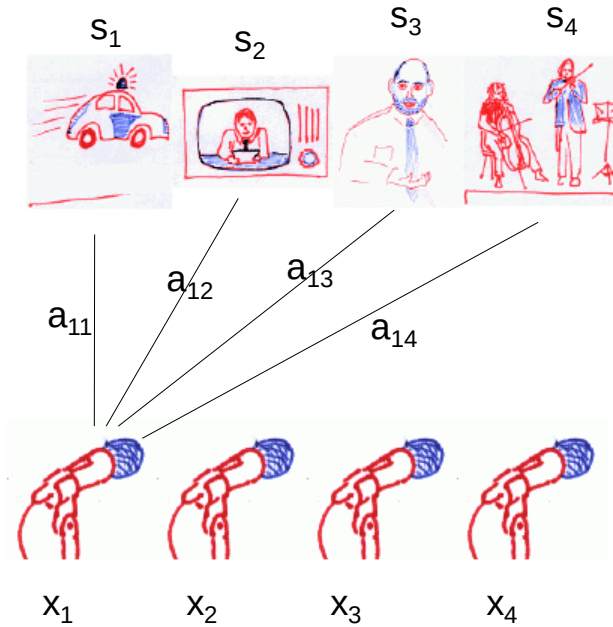


- Cocktail party or Blind Source Separation (BSS) problem
 - Ill posed problem, unless assumptions are made!
- Most common assumption is that source signals are statistically independent. This means knowing value of one of them gives no information about the other.
- Methods based on this assumption are called Independent Component Analysis methods
 - statistical techniques for decomposing a complex data set into independent parts.
- It can be shown that under some reasonable conditions, if the ICA assumption holds, then the source signals can be recovered up to permutation and scaling.

Source Separation Using ICA



The ICA model

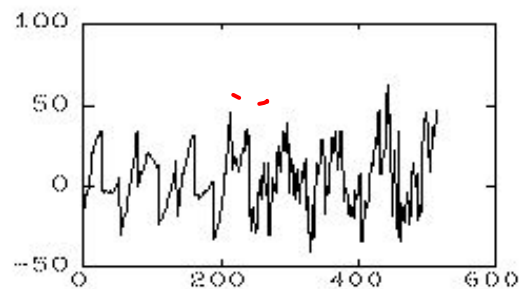
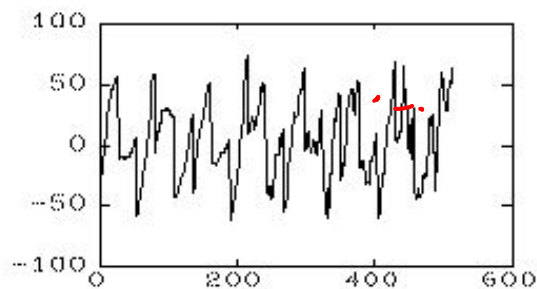
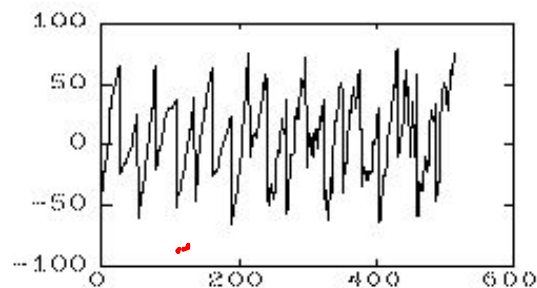
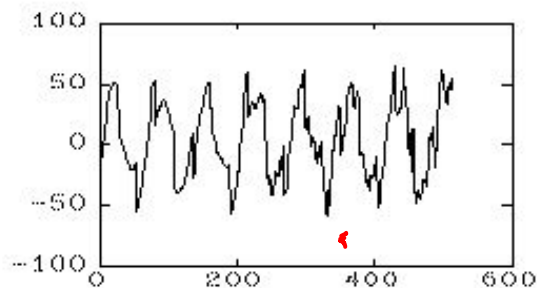


$$x_i(t) = a_{i1} * s_1(t) + a_{i2} * s_2(t) + a_{i3} * s_3(t) + a_{i4} * s_4(t)$$

Here, $i=1:4$.

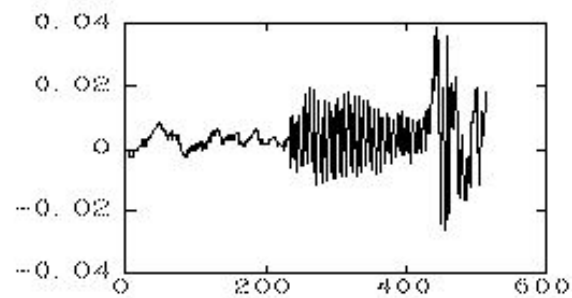
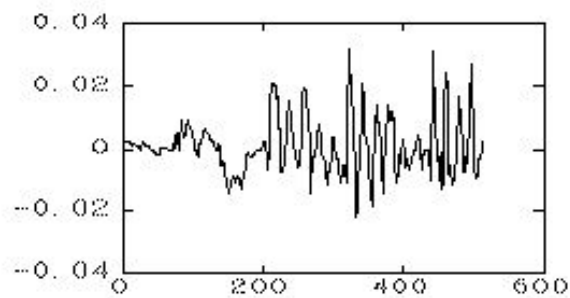
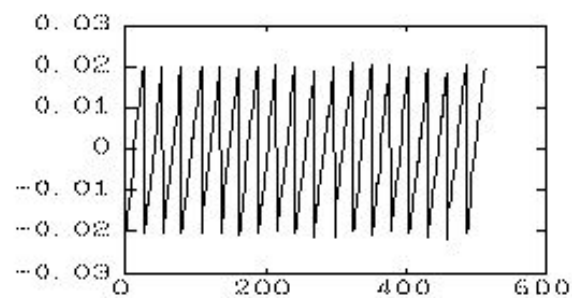
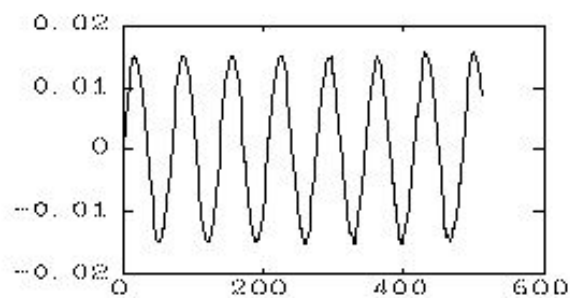
In vector-matrix notation, and dropping index t , this is

$$\mathbf{x} = \mathbf{A} * \mathbf{s}$$



This is recorded by the microphones: a linear mixture of the sources

$$x_i(t) = a_{i1} * s_1(t) + a_{i2} * s_2(t) + a_{i3} * s_3(t) + a_{i4} * s_4(t)$$



Recovered signals

Problem: Determine the source signals $\mathbf{s}$, given **only** the mixtures $\mathbf{x}$.

- If we knew the mixing parameters a_{ij} then we would just need to solve a linear system of equations.
- We know neither a_{ij} nor s_i .
- ICA was initially developed to deal with problems closely related to the cocktail party problem
- Later it became evident that ICA has many other applications
 - e.g., from electrical recordings of brain activity from different locations of the scalp (EEG signals) recover underlying components of brain activity

ICA Solution and Applicability



- ICA is a statistical method, the goal of which is to decompose given multivariate data into a linear sum of statistically independent components.
- For example, given two-dimensional vector, $\mathbf{x} = [x_1 \ x_2]^T$, ICA aims at finding the following decomposition

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} s_1 + \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} s_2$$

$$\mathbf{x} = \mathbf{a}_1 s_1 + \mathbf{a}_2 s_2$$

where $\mathbf{a}_1, \mathbf{a}_2$ are basis vectors and s_1, s_2 are basis coefficients.

- Constraint: Basis coefficients s_1 and s_2 are statistically independent.

Application domains of ICA

- Blind source separation
- Image denoising
- Medical signal processing – fMRI, ECG, EEG
- Modelling of the hippocampus and visual cortex
- Feature extraction, face recognition
- Compression, redundancy reduction
- Watermarking
- Clustering
- Time series analysis (stock market, microarray data)
- Topic extraction
- Econometrics: Finding hidden factors in financial data

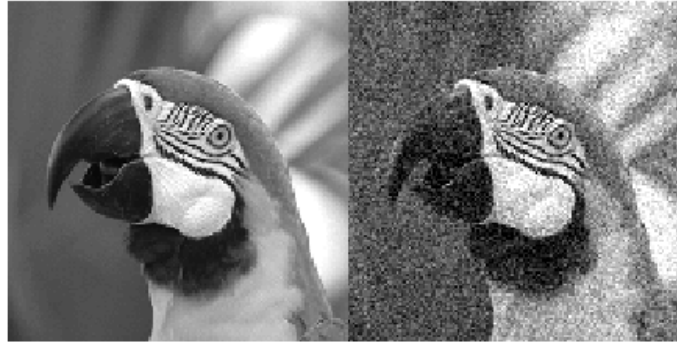
Image denoising

innovate

achieve

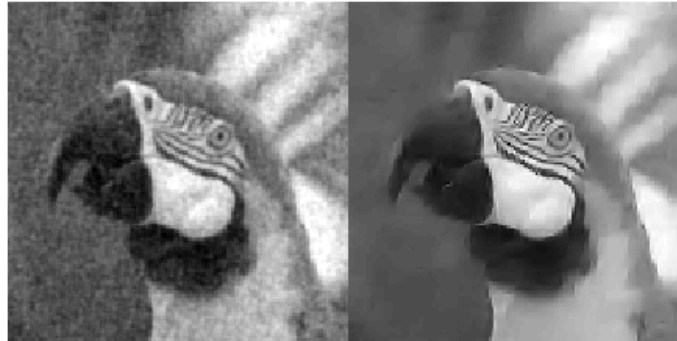
lead

Original
image



Noisy
image

Wiener
filtering



ICA
filtering

Approaches to ICA



- Unsupervised Learning
 - Factorial coding (Minimum entropy coding, Redundancy reduction)
 - Maximum likelihood learning
 - Nonlinear information maximization (entropy maximization)
 - Negentropy maximization
 - Bayesian learning
- Statistical Signal Processing
 - Higher-order moments or cumulants
 - Joint approximate diagonalization
 - Maximum likelihood estimation

Factorial Faces for Recognition



Eigenfaces

PCA/FA

It finds a linear data representation that best model the covariance structure of face image data (**second-order relations**)

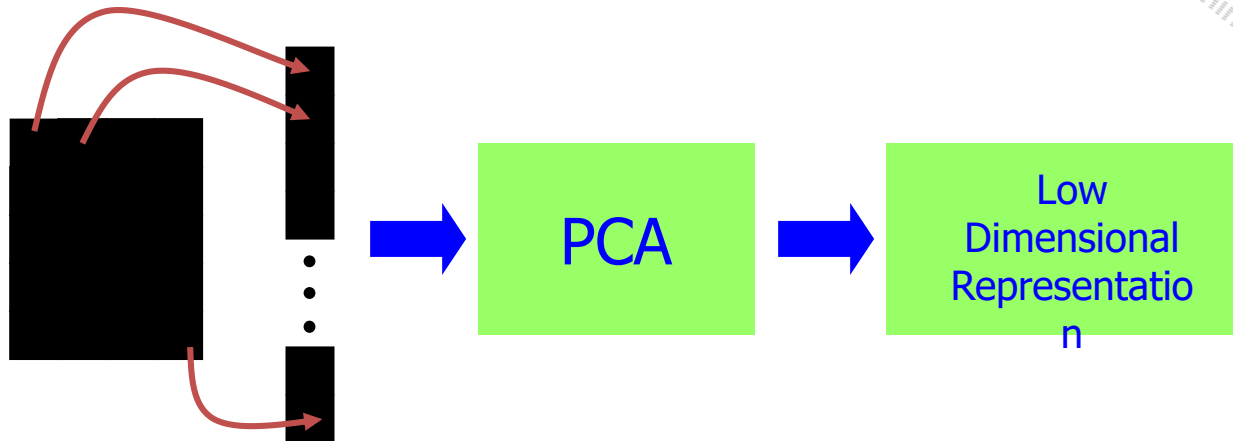
Factorial Faces

Factorial Coding/ICA

It finds a linear data representation that Best model the probability distribution of Face image data (**high-order relations**)

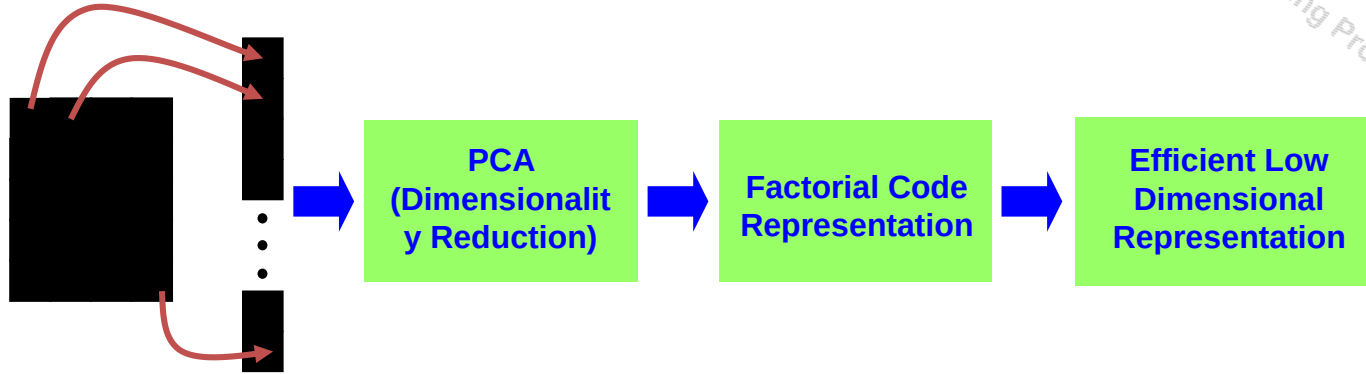
(IEEE BMCV 2000)

Eigenfaces



$$\text{Face Image} = a_1 \times \text{Eigenface}_1 + a_2 \times \text{Eigenface}_2 + a_3 \times \text{Eigenface}_3 + a_4 \times \text{Eigenface}_4 + \dots + a_n \times \text{Eigenface}_n$$

Factorial Code Representation: Factorial Faces



$$\text{Face Image} = b_1 \times \text{Feature 1} + b_2 \times \text{Feature 2} + b_3 \times \text{Feature 3} + b_4 \times \text{Feature 4} + \dots + b_n \times \text{Feature n}$$

Experimental Results



Figure 1. Sample images in the training set.

(neutral expression, anger, and right-light-on from first session;
smile and left-light-on from second session)



Figure 2. Sample images in the test set.

(smile and left-light-on from first session; neutral expression,
anger, and right-light-on from second session)

Eigenfaces vs Factorial Faces

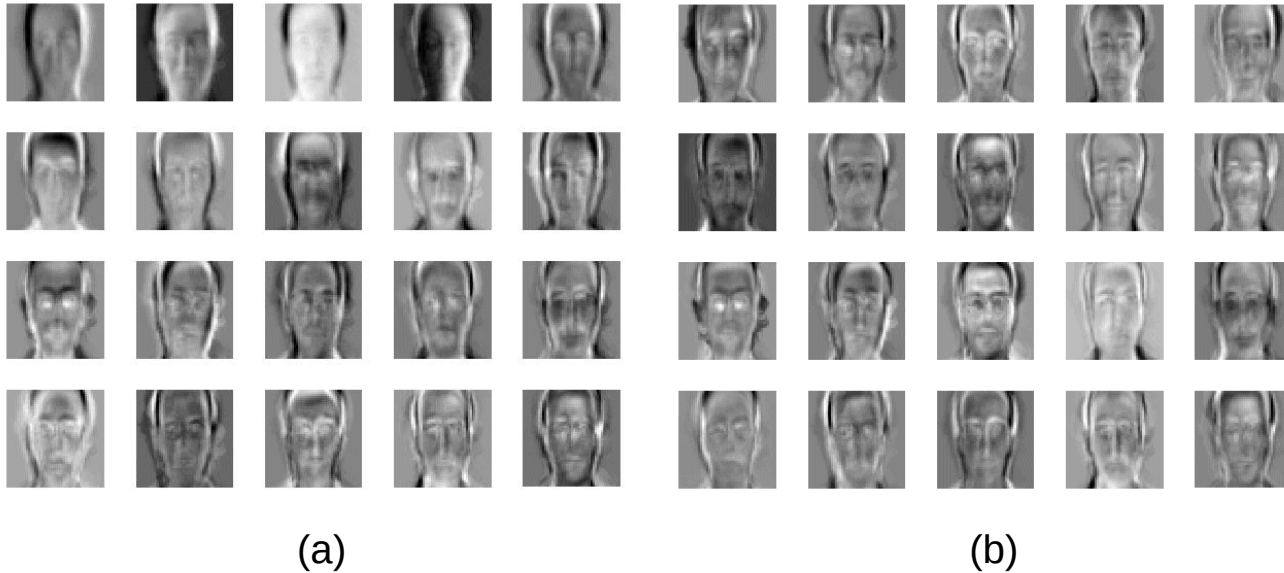


Figure 3. First 20 basis images: (a) in eigenface method; (b) factorial code. They are ordered by column, then, by row.

Experimental Results

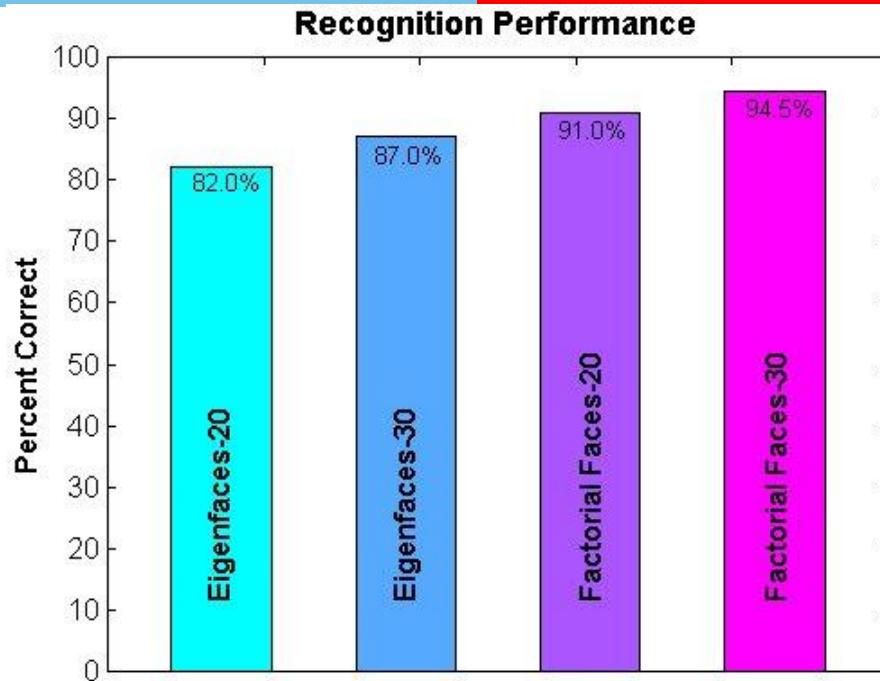


Figure 1. The comparison of recognition performance using Nearest Neighbor: the eigenface method and Factorial Code Representation using 20 or 30 principal components in the snap-shot method;

Experimental Results

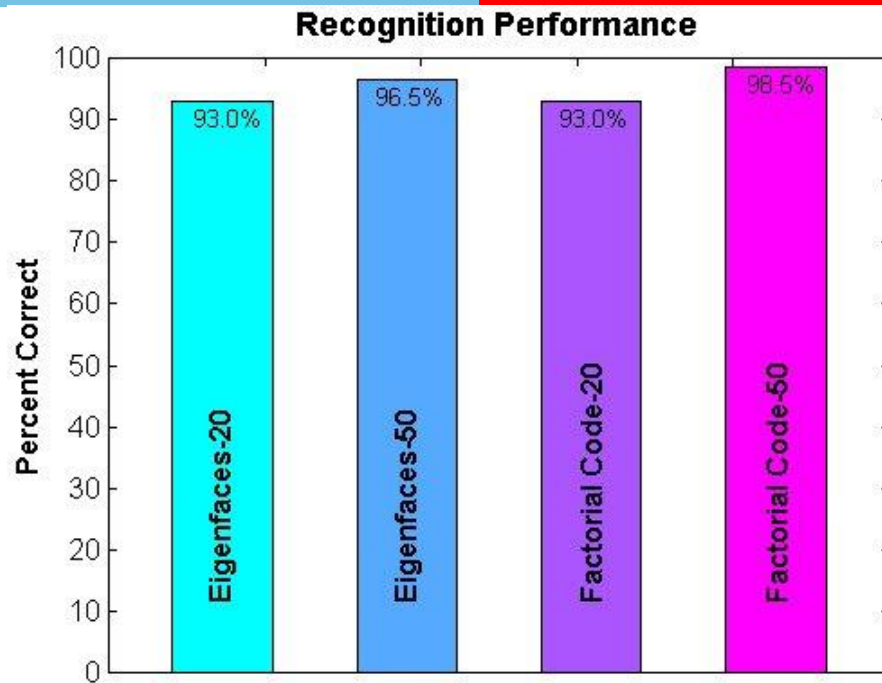


Figure 2. The comparison of recognition performance using MLP Classifier: the eigenface method and Factorial Code Representation using 20 or 50 principal components in the snap-shot method;

PCA vs ICA



- Linear Transform
 - Compression
 - Classification
- PCA
 - Focus on uncorrelated and Gaussian components
 - Second-order statistics
 - Orthogonal transformation
- ICA
 - Focus on independent and non-Gaussian components
 - Higher-order statistics
 - Non-orthogonal transformation

Gaussians and ICA



- If some components are gaussian and some are non-gaussian.
 - Can estimate all non-gaussian components
 - Linear combination of gaussian components can be estimated.
 - If only one gaussian component, model can be estimated
- ICA sometimes viewed as non-Gaussian factor analysis

Canonical Correlation Analysis



- Measuring the linear relationship between two multi dimensional variables
- Finding two sets of basis vectors such that the correlation between the projections of the variables onto these basis vectors is maximized
- Determine Correlation Coefficients

Canonical Correlation Analysis



- More than one canonical correlations will be found each corresponding to a different set of basis vectors/Canonical variates
- Correlations between successively extracted canonical variates are smaller and smaller
- Correlation coefficients : Proportion of correlation between the canonical variates accounted for by the particular variable.

Differences with Correlation

- Not dependent on the coordinate system of variables
- Finds direction that yield maximum correlations

Application



- Kernel CCA can be used to find non linear relationships between multi variates
- Two views of the same semantic object to extract the representation of the semantics
 - Speaker Recognition – Audio and Lip movement
 - Image retrieval – Image features (HSV, Texture) and Associated text

Use of KCCA in cross-modal retrieval



- 400 records of JPEG images for each class with associated text and a total of 3 classes
- Data was split randomly into 2 parts for training and test
- Features
 - Image – HSV Color, Gabor texture
 - Text – Term frequencies
- Results were taken for an average of 10 runs

Aviation				
Sports				
Paintball				

Figure 1: Example of images in database.

Cross-modal retrieval



- Content based retrieval: Retrieve images in the same class tested with 10 and 30 images sets
 - where $\text{count}^j_k = 1$ if the image k in the set is of the same label as the text query present in the set, else $\text{count}^j_k = 0$.

$$\text{success \% for image set } i = \frac{\sum_{j=1}^{600} \sum_{k=1}^i \text{count}^j_k}{i \times 600} \times 100,$$

Comparison of KCCA (with 5 and 30 Eigen vectors) with GVSM



Table 1: Success Cross-Results Between Kernel-CCA and Generalized Vector Space.

Image Set	GVSM Success	KCCA Success (30)	KCCA Success (5)
10	78.93%	85%	90.97%
30	76.82%	83.02%	90.69%

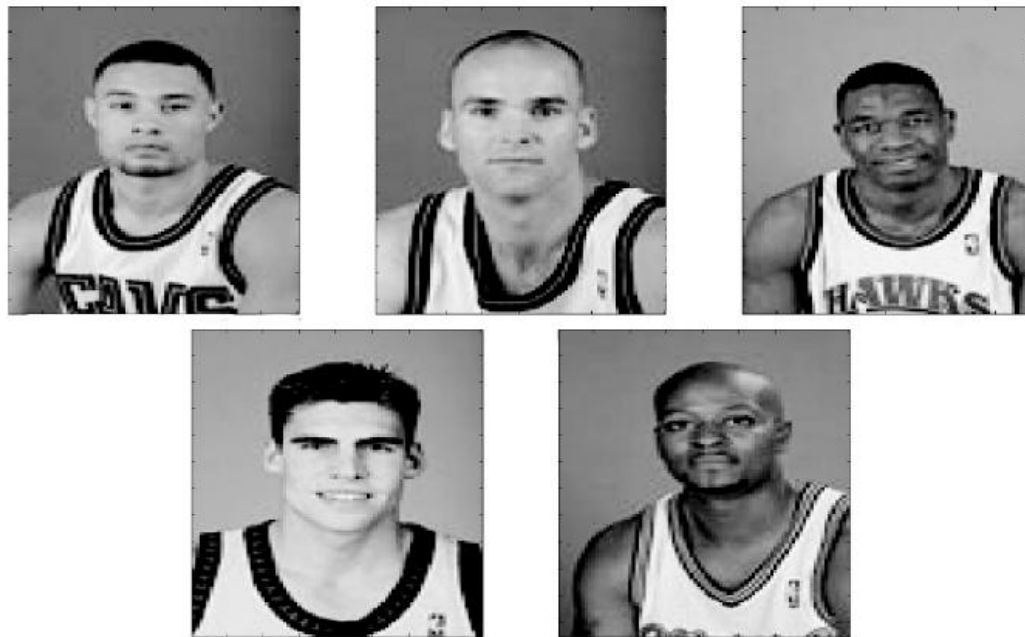


Figure 2: Images retrieved for the text query: "height: 6-11 weight: 235 lbs position: forward born: september 18, 1968, split, croatia college: none."

Mate based retrieval

- Match the exact image among the selected retrieved images
- Tested with 10 and 30 images sets
 - where $\text{count}_j = 1$ if the exact matching image was present in the set else it is 0

$$\text{success \% for image set } i = \frac{\sum_{j=1}^{600} \text{count}_j}{600} \times 100,$$



Figure 5: Images retrieved for the text query: "at phoenix sky harbor on july 6, 1997. 757-2s7, n907wa phoenix suns taxis past n902aw teamwork america west america west 757-2s7, n907wa phoenix suns taxis past n901aw arizona at phoenix sky harbor on july 6, 1997." The actual match is the middle picture in the top row.

Comparison of KCCA (with 30 and 150 Eigen vectors) with GVSM Mate based retrieval



Table 3: Success Cross-Results Between Kernel-CCA and Generalised Vector Space.

Image Set	GVSM Success	KCCA Success (30)	KCCA Success (150)
10	8%	17.19%	59.5%
30	19%	32.32%	69%

Comments



- The good
 - Good explanation of CCA and KCCA
 - Innovative use of KCCA in image retrieval application
- The bad
 - The data set and the number of classes used were small
 - The image set size is not taken into account while calculating accuracy in Mate based retrieval
 - Could have done cross-validation tests

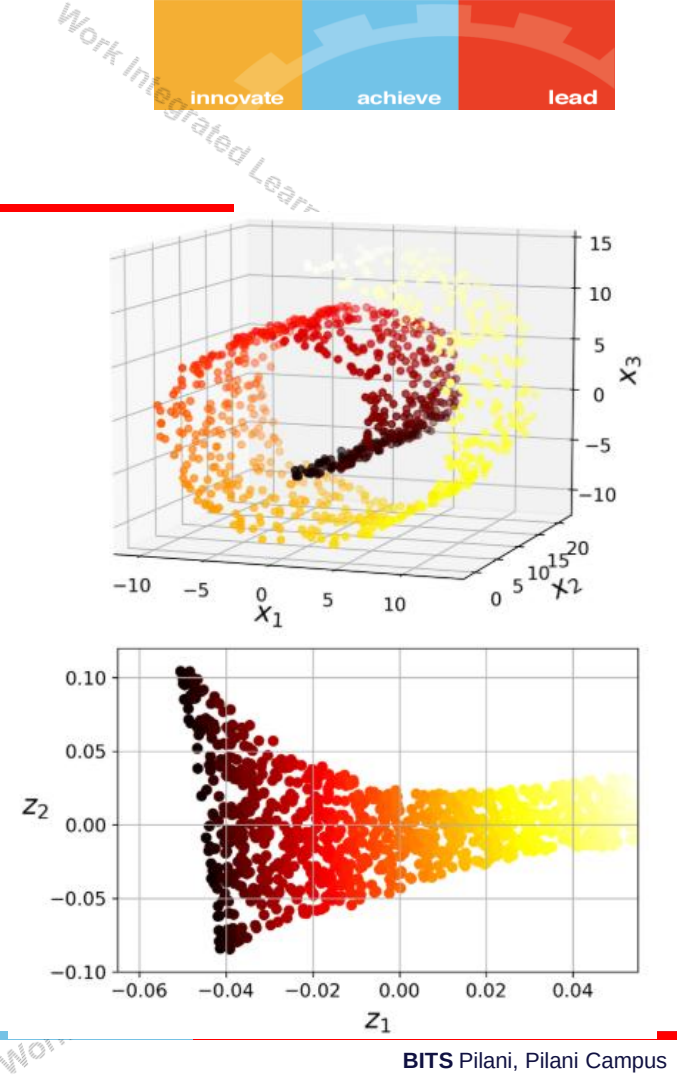
Limitations and Assumptions of CCA



- At least 40 to 60 times as many cases as variables is recommended to get reliable estimates for two roots
 - Barcikowski & Stevens (1986)
- Outliers can greatly affect the canonical correlation
- Variables in two sets should not be completely redundant

Locally Linear Embedding (LLE)

- Nonlinear dimensionality reduction (NLDR) technique.
- Manifold Learning technique that does not rely on projections.
- Works by first measuring how each training instance linearly relates to its closest neighbors, and then looking for a low-dimensional representation of the training set where these local relationships are best preserved.
- Particularly good at unrolling twisted manifolds, especially when there is not too much noise.



LLE (contd.)

Here's how LLE works: for each training instance $\mathbf{x}^{(i)}$, the algorithm identifies its k closest neighbors (in the preceding code $k = 10$), then tries to reconstruct $\mathbf{x}^{(i)}$ as a linear function of these neighbors. More specifically, it finds the weights $w_{i,j}$ such that the squared distance between $\mathbf{x}^{(i)}$ and $\sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)}$ is as small as possible, assuming $w_{i,j} = 0$ if $\mathbf{x}^{(i)}$ is not one of the k closest neighbors of $\mathbf{x}^{(i)}$. Thus the first step of LLE is the constrained optimization problem described in **Equation 8-4**, where $\mathbf{W}$ is the weight matrix containing all the weights $w_{i,j}$. The second constraint simply normalizes the weights for each training instance $\mathbf{x}^{(i)}$.

Equation 8-4. LLE step one: linearly modeling local relationships

$$\begin{aligned} \hat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \quad & \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2 \\ \text{subject to} \quad & \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ c.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^m w_{i,j} = 1 & \text{for } i = 1, 2, \dots, m \end{cases} \end{aligned}$$

LLE (contd.)

After this step, the weight matrix $\hat{\mathbf{W}}$ (containing the weights $\hat{w}_{i,j}$) encodes the local linear relationships between the training instances. The second step is to map the training instances into a d -dimensional space (where $d < n$) while preserving these local relationships as much as possible. If $\mathbf{z}^{(i)}$ is the image of $\mathbf{x}^{(i)}$ in this d -dimensional space, then we want the squared distance between $\mathbf{z}^{(i)}$ and $\sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)}$ to be as small as possible. This idea leads to the unconstrained optimization problem described in Equation 8-5. It looks very similar to the first step, but instead of keeping the instances fixed and finding the optimal weights, we are doing the reverse: keeping the weights fixed and finding the optimal position of the instances' images in the low-dimensional space. Note that $\mathbf{Z}$ is the matrix containing all $\mathbf{z}^{(i)}$.

Equation 8-5. LLE step two: reducing dimensionality while preserving relationships

$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$

Scikit-Learn's LLE implementation has the following computational complexity: $O(m \log(m)n \log(k))$ for finding the k nearest neighbors, $O(mnk^3)$ for optimizing the weights, and $O(dm^2)$ for constructing the low-dimensional representations. Unfortunately, the m^2 in the last term makes this algorithm scale poorly to very large datasets.

t-Distributed Stochastic Neighbor Embedding (t-SNE)

innovate

achieve

lead

More Details

Reduces dimensionality while trying to keep similar instances close and dissimilar instances apart. It is mostly used for visualization, in particular to visualize clusters of instances in high-dimensional space (e.g., to visualize the MNIST images in 2D).

- Models a point being selected as a neighbor of another point in both higher and lower dimensions. Starts by calculating a pairwise similarity between all data points in the high-d space using a Gaussian kernel. Points far apart are less probable of being picked than closer together points.
- Then, tries to map higher-D data points onto lower-D space while preserving the pairwise similarities.
- By minimizing the divergence between the P.D. of the original high-d and lower-D. Uses gradient descent to minimize the divergence. The lower-D embedding optimized to a stable state.

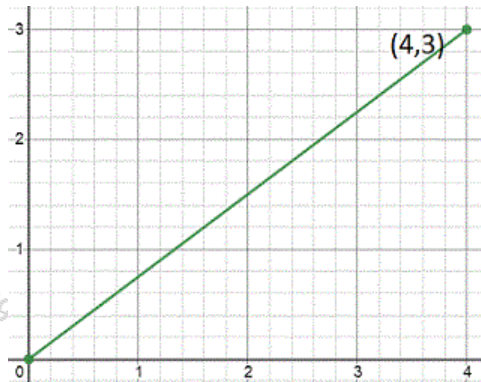


APPENDIX

PCA : Some Mathematics

Scalar: Scalar is nothing but a number with a value. e.g. 5 is a scalar, 2.45 is a scalar.

Vector: Vector can be thought of an object with a magnitude and a direction. A coordinate in two dimension is an example of a vector. e.g. (4,3).



$$\text{Distance} = \sqrt{4^2 + 3^2} = \sqrt{25} = 5$$

PCA: Some Mathematics

Matrix: Matrix is an array of numbers. Matrix may be square or rectangular shape. Note that all vectors can be represented as matrix. In the example above our vector can be represented as matrix as shown in the figure below:

$$\begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

2×1 Matrix

1.4 Determinant: It represents a number which can be calculated from a square matrix. e.g. if A is a square matrix then |A| represents determinant of A. Determinant of a 2x2 matrix can be calculated as below:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$\text{Hence, } |A| = (1 \times 4) - (3 \times 2) = 4 - 6 = -2$$

PCA: Some Mathematics

Identity Matrix: Identity matrix (denoted as I) is a square matrix where the diagonal elements of the matrix are 1 and rest of the elements are 0.

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Invertible matrix: Z is said to be invertible matrix (it should be square matrix) if it satisfy the following equation:

$$Z^{-1}Z = Z Z^{-1} = I$$

where Z^{-1} is the inverse matrix of Z , I is the identity matrix.

Note: Determinant of Z , that is $|Z| \neq 0$ if and only if Z is not invertible.

PCA: Some Mathematics

Matrix Multiplication:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$$AB = \begin{bmatrix} 1 \times 5 + 2 \times 6 \\ 3 \times 5 + 4 \times 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

Eigenvector: Eigenvector is a non-zero vector (say, v) which when multiplied with another square matrix (say, A) produces another matrix (also, a vector) which is an scalar multiple of the original vector v .

$$Av = \lambda v$$

PCA: Some Mathematics

Eigenvalue: In the above equation λ is called the eigenvalue of the eigenvector v .

$$Av = \lambda v$$

$$Av = \lambda v$$

$$\Rightarrow Av - \lambda v = 0$$

$$\Rightarrow (A - \lambda I)v = 0$$

Here I is an Identity matrix with a dimension equal to that of A .

PCA: Some Statistics

Let $X = (X_1, X_2, \dots, X_n)$ be a list of numbers.

$\bar{X}$ is the mean of the numbers, and $\bar{X} = \frac{\sum_{i=1}^n X_i}{n}$

Standard deviation (S) is the measure of dispersion of data from the mean in a list of numbers. The formula to calculate the standard deviation is:

$$s = \sqrt{\frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n-1}}$$

Variance is the square of the standard deviation.

$$\text{var}(X) = s^2 = \frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n-1}$$

PCA: Some Mathematics

Covariance: As variance depicts spread of data in one column, covariance is the joint form (related to two columns in a dataset) of the same thing.

$$\text{var}(X) = \text{cov}(X, X) = \frac{\sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})}{n-1}$$

$$\text{cov}(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{n-1}$$

PCA: Some Mathematics

Z-Score: We can standardize or scale a group of observed values using **z-score** for which mean is deducted from each observed value and then divided by the standard deviation. The formula is:

$$Z = \frac{X_i - \bar{X}}{S_x}$$

X_i = Observed value

$\bar{X}$ = Mean of the group

S_x = Standard Deviation of the group

PCA: Some Mathematics

X	Y
2	3
4	5
6	5
6	7
7	8
5	8

$$\text{Mean (X)} = \bar{X} = (2+4+6+6+7+5)/6 = 5$$

$$\text{Mean (Y)} = \bar{Y} = (3+5+5+7+8+8)/6 = 6$$

$$S_x = \sqrt{((2-5)^2 + (4-5)^2 + (6-5)^2 + (6-5)^2 + (7-5)^2 + (5-5)^2)/6}$$

$$= \sqrt{(9 + 1 + 1 + 1 + 4 + 0)/6}$$

$$= \sqrt{16/6} = 1.633$$

$$S_y = \sqrt{((3-6)^2 + (5-6)^2 + (5-6)^2 + (7-6)^2 + (8-6)^2 + (8-6)^2)/6}$$

$$= \sqrt{(9 + 1 + 1 + 1 + 4 + 4)/6}$$

$$= \sqrt{20/6} = 1.826$$

S_x and S_y are the standard deviations of column X and Y respectively.

PCA: Some Mathematics

Original data

X	Y
2	3
4	5
6	5
6	7
7	8
5	8

Z-Score calculation

X	Y
$(2-5)/1.633$	$(3-6)/1.826$
$(4-5)/1.633$	$(5-6)/1.826$
$(6-5)/1.633$	$(5-6)/1.826$
$(6-5)/1.633$	$(7-6)/1.826$
$(7-5)/1.633$	$(8-6)/1.826$
$(5-5)/1.633$	$(8-6)/1.826$

=

Scaled data

X	Y
-1.837	-1.643
-0.612	-0.548
0.612	-0.548
0.612	0.548
1.225	1.095
0	1.095

PCA: Some Mathematics

Original data

X	Y
2	3
4	5
6	5
6	7
7	8
5	8

Z-Score calculation

X	Y
(2-5)/1.633	(3-6)/1.826
(4-5)/1.633	(5-6)/1.826
(6-5)/1.633	(5-6)/1.826
(6-5)/1.633	(7-6)/1.826
(7-5)/1.633	(8-6)/1.826
(5-5)/1.633	(8-6)/1.826

Scaled data

X	Y
-1.837	-1.643
-0.612	-0.548
0.612	-0.548
0.612	0.548
1.225	1.095
0	1.095

after Z scaling, the mean of X and Y becomes 0, i.e. $\bar{X} = 0, \bar{Y} = 0$

$$\text{Cov}(X,X) = \frac{\sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})}{n-1} = \frac{\sum_{i=1}^n X_i^2}{n-1}$$

$$= ((-1.837)^2 + (-0.612)^2 + (0.612)^2 + (0.612)^2 + (1.225)^2 + 0)/5$$

$$= 1.2$$

PCA: Some Mathematics

Original data

X	Y
2	3
4	5
6	5
6	7
7	8
5	8

Z-Score calculation

X	Y
(2-5)/1.633	(3-6)/1.826
(4-5)/1.633	(5-6)/1.826
(6-5)/1.633	(5-6)/1.826
(6-5)/1.633	(7-6)/1.826
(7-5)/1.633	(8-6)/1.826
(5-5)/1.633	(8-6)/1.826

=

Scaled data

X	Y
-1.837	-1.643
-0.612	-0.548
0.612	-0.548
0.612	0.548
1.225	1.095
0	1.095

$$\text{Cov}(Y,Y) = \frac{\sum_{i=1}^n (Y_i - \bar{Y})(Y_i - \bar{Y})}{n-1} = \frac{\sum_{i=1}^n Y_i^2}{n-1}$$

$$= ((-1.643)^2 + (-0.548)^2 + (-0.548)^2 + (0.548)^2 + (1.095)^2 + (1.095)^2) / 5$$

$$= 1.2$$

PCA: Some Mathematics

Original data

X	Y
2	3
4	5
6	5
6	7
7	8
5	8

Z-Score calculation

X	Y
$(2-5)/1.633$	$(3-6)/1.826$
$(4-5)/1.633$	$(5-6)/1.826$
$(6-5)/1.633$	$(5-6)/1.826$
$(6-5)/1.633$	$(7-6)/1.826$
$(7-5)/1.633$	$(8-6)/1.826$
$(5-5)/1.633$	$(8-6)/1.826$

Scaled data

X	Y
-1.837	-1.643
-0.612	-0.548
0.612	-0.548
0.612	0.548
1.225	1.095
0	1.095

$$\text{Cov}(X,Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{n-1} = \frac{\sum_{i=1}^n X_i Y_i}{n-1}$$

$$= ((-1.837)(-1.643) + (-0.612)(-0.548) + (0.612)(-0.548) + (0.612)(0.548) + (1.225)(1.095) + 0)/5$$

$$= 0.939$$

PCA: Some Mathematics

We know, $\text{Cov}(Y, X) = \text{Cov}(X, Y)$

We know covariance matrix of X, Y is

$$\begin{aligned}\text{Covariance Matrix} &= \begin{bmatrix} \text{Cov}(X, X) & \text{Cov}(X, Y) \\ \text{Cov}(Y, X) & \text{Cov}(Y, Y) \end{bmatrix} \\ &= \begin{bmatrix} 1.2 & 0.939 \\ 0.939 & 1.2 \end{bmatrix}\end{aligned}$$

PCA: Some Mathematics

From, above, if λ is the Eigen value of given matrix A then

$$|A - \lambda I| = 0$$

Now, the Eigen value of our covariance matrix will be

$$A - \lambda I = \begin{bmatrix} 1.2 & 0.939 \\ 0.939 & 1.2 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1.2 & 0.939 \\ 0.939 & 1.2 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} = \begin{bmatrix} 1.2 - \lambda & 0.939 \\ 0.939 & 1.2 - \lambda \end{bmatrix}$$

$$\text{So, } \begin{vmatrix} 1.2 - \lambda & 0.939 \\ 0.939 & 1.2 - \lambda \end{vmatrix} = 0$$

$$\Rightarrow (1.2 - \lambda)(1.2 - \lambda) - (0.939) \times (0.939) = 0$$

$$\Rightarrow 1.44 - (2 \times 1.2) \lambda + \lambda^2 - 0.8817 = 0$$

$$\Rightarrow \lambda^2 - 2.4 \lambda + 0.5583 = 0$$

Solving for λ we get $\lambda = 2.139$ and $\lambda = 0.261$

So, two Eigen values are 2.139 and 0.261

PCA: Some Mathematics

For the vector to be Eigen vector the length of the vector should be 1.

Using Pythagoras theorem, distance $= \sqrt{1^2 + 1^2} = \sqrt{2}$

Now if we divide the vector, represented as matrix $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ by $\sqrt{2}$, its length will become 1 and we will get our desired Eigen vector.

$$\text{Eigen Vector} = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}$$

PCA: Some Mathematics

The first feature vector (-1.837, -1.643) will be transformed using the following formula:

(Transpose of Eigen vector) X (Feature vector)

Let's put the corresponding values:

$$\begin{aligned}
 & \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}^T \times \begin{bmatrix} -1.837 \\ -1.643 \end{bmatrix} \\
 &= [0.7071 \quad 0.7071] \begin{bmatrix} -1.837 \\ -1.643 \end{bmatrix} \\
 &= 0.7071 \times (-1.837) + 0.7071 \times (-1.643) \\
 &= -(1.2989 + 1.1617) \\
 &= -2.461
 \end{aligned}$$

Projected Data
-2.461
-0.820
0.046
0.820
1.640
0.774

Hence, $\begin{bmatrix} -1.837 \\ -1.643 \end{bmatrix}$ is projected to = -2.461

Eigenvector and Eigenvalue

$$Ax = \lambda x$$

A: Square Matrix

λ : Eigenvector or characteristic vector

X: Eigenvalue or characteristic value



- *The zero vector can not be an eigenvector*
- *The value zero can be eigenvalue*



Eigenvector and Eigenvalue

$$Ax = \lambda x$$

A: Square Matrix

λ : Eigenvector or characteristic vector

X: Eigenvalue or characteristic value

Example

Show $x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ is an eigenvector for $A = \begin{bmatrix} 2 & -4 \\ 3 & -6 \end{bmatrix}$

$$\text{Solution : } Ax = \begin{bmatrix} 2 & -4 \\ 3 & -6 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\text{But for } \lambda = 0, \lambda x = 0 \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Thus, x is an eigenvector of A , and $\lambda = 0$ is an eigenvalue. BITS Pilani, Pilani Campus

Eigenvector and Eigenvalue

innovate

achieve

lead

$$Ax = \lambda x$$



$$Ax - \lambda x = 0$$
$$(A - \lambda I)x = 0$$

If we define a new matrix B:



$$B = A - \lambda I$$
$$Bx = 0$$

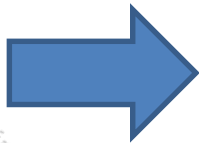
If B has an inverse:



$$x = B^{-1}0 = 0$$



**BUT! an
eigenvector
cannot be zero!!**



x will be an eigenvector of A if and only if
B does not have an inverse, or equivalently
 $\det(B)=0$:

$$\det(A - \lambda I) = 0$$

Eigenvector and Eigenvalue

innovate

achieve

lead

Example 1: Find the eigenvalues of

$$A = \begin{bmatrix} 2 & -12 \\ 1 & -5 \end{bmatrix}$$

$$\begin{aligned} |\lambda I - A| &= \begin{vmatrix} \lambda - 2 & 12 \\ -1 & \lambda + 5 \end{vmatrix} = (\lambda - 2)(\lambda + 5) + 12 \\ &= \lambda^2 + 3\lambda + 2 = (\lambda + 1)(\lambda + 2) \end{aligned}$$

two eigenvalues: $-1, -2$

Note: The roots of the characteristic equation can be repeated. That is, $\lambda_1 = \lambda_2 = \dots = \lambda_k$. If that happens, the eigenvalue is said to be of multiplicity k .

Example 2: Find the eigenvalues of

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

$$|\lambda I - A| = \begin{vmatrix} \lambda - 2 & -1 & 0 \\ 0 & \lambda - 2 & 0 \\ 0 & 0 & \lambda - 2 \end{vmatrix} = (\lambda - 2)^3 = 0$$

$\lambda = 2$ is an eigenvalue of multiplicity 3.



PCA: one attribute first

Question: how much spread is in the data along the axis?
(distance to the mean)

Variance=Standard deviation²

$$s^2 = \frac{\sum_{i=1}^n (X_i - \bar{X})^2}{(n-1)}$$

Temperature
42
40
24
30
15
18
15
30
15
30
35
30
40
30



Now consider two dimensions

Covariance: measures the correlation between X and Y

- $\text{Cov}(X,Y)=0$: independent
- $\text{Cov}(X,Y)>0$: move same dir
- $\text{Cov}(X,Y)<0$: move oppo dir

$$\text{cov}(X,Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{(n-1)}$$

X=Temperature	Y=Humidity
40	90
40	90
40	90
30	90
15	70
15	70
15	70
30	90
15	70
30	70
30	70
30	90
40	70

More than two attributes: covariance matrix



Contains covariance values between all possible dimensions (=attributes):

$$C^{n \times n} = (c_{ij} \mid c_{ij} = \text{cov}(Dim_i, Dim_j))$$

Example for three attributes (x,y,z):

$$C = \begin{pmatrix} \text{cov}(x, x) & \text{cov}(x, y) & \text{cov}(x, z) \\ \text{cov}(y, x) & \text{cov}(y, y) & \text{cov}(y, z) \\ \text{cov}(z, x) & \text{cov}(z, y) & \text{cov}(z, z) \end{pmatrix}$$

Transformed Data

- Eigenvalues λ_j corresponds to variance on each component j
- Thus, sort by λ_j
- Take the first p eigenvectors $\mathbf{e}_i$, where p is the number of top eigenvalues
- These are the directions with the largest variances

$$\begin{pmatrix} y_{i1} \\ y_{i2} \\ \dots \\ y_{ip} \end{pmatrix} = \begin{pmatrix} e_1 \\ e_2 \\ \dots \\ e_p \end{pmatrix} \begin{pmatrix} x_{i1} - \bar{x}_1 \\ x_{i2} - \bar{x}_2 \\ \dots \\ x_{in} - \bar{x}_n \end{pmatrix}$$

Eigenvalues & eigenvectors



Vectors $\mathbf{x}$ having same direction as $A\mathbf{x}$ are called *eigenvectors* of A (A is an n by n matrix).

In the equation $A\mathbf{x}=\lambda\mathbf{x}$, λ is called an *eigenvalue* of A .

$$\begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} \mathbf{x} \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 8 \end{pmatrix} = 4 \mathbf{x} \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

Eigenvalues & eigenvectors

$$A\mathbf{x}=\lambda\mathbf{x} \Leftrightarrow (A-\lambda I)\mathbf{x}=0$$

How to calculate $\mathbf{x}$ and λ :

- Calculate $\det(A-\lambda I)$, yields a polynomial (degree n)
- Determine roots to $\det(A-\lambda I)=0$, roots are eigenvalues λ
- Solve $(A-\lambda I)\mathbf{x}=0$ for each λ to obtain eigenvectors $\mathbf{x}$

PCA



1. principal component (PC1)

- The eigenvalue with the largest absolute value will indicate that the data have the largest variance along its eigenvector, the direction along which there is greatest variation

2. principal component (PC2)

- the direction with maximum variation left in data, orthogonal to the 1. PC

In general, only few directions manage to capture most of the variability in the data.

Steps of PCA



Let $\bar{X}$ be the mean vector (taking the mean of all rows)

Adjust the original data by the mean

$$X' = X - \bar{X}$$

Compute the covariance matrix C of adjusted X

Find the eigenvectors $\bar{x}$ and eigenvalues of C .

For matrix C , vectors e (=column vector) having same direction as Ce :

- *eigenvectors* of C is e such that $Ce = \lambda e$,
- λ is called an *eigenvalue* of C .

$$Ce = \lambda e \Leftrightarrow (C - \lambda I)e = 0$$

- Most data mining packages do this for you.

Eigenvalues

innovate

achieve

lead

Calculate eigenvalues λ and eigenvectors $\mathbf{x}$ for covariance matrix:

- Eigenvalues λ_j are used for calculation of [% of total variance] (V_j) for each component j :

$$V_j = 100 \cdot \frac{\lambda_j}{\sum_{x=1}^n \lambda_x}$$

$$\sum_{x=1}^n \lambda_x = n$$